

GUIDE DE DEPLOIEMENT DE L'APPLICATION DE MEGASOFT

Objectifs

Ce guide représente la marche à suivre pour le déploiement de l'application web `Tomcat` connecté à une base de données `MySQL` en utilisant la technologie de conteneurisation Docker.

Résultats

Le résultat que vous obtiendrez sera un environnement constitué de deux conteneurs dont :

- Un système d'exploitation Linux plus précisément `Ubuntu v23.04` dans lequel sera installé votre application `Tomcat` ainsi que toutes ses dépendances ;
- Un serveur de base de données `MySQL v8.0.33`.

Tous ces conteneurs seront dans un même réseau.

1. Modification du projet

Dans le repertoire racine du projet en local:

- Créer le dossier `data` dans lequel vous mettrez le script `sql` de la base de donnees ainsi que le fichier `Dockerfile` pour la création de l'image de la BD. Ce fichier se présente comme suit :

```
1 FROM mysql
2 COPY . /docker-entrypoint-initdb.d
3 ENV MYSQL_ROOT_PASSWORD=manager10
4 ENV MYSQL_DATABASE=bdrsupreprodv1
```

- Créez le dossier `database` qui va assurer la persistance des données de la BD conteneurisée.
- Créez le fichier `Dockerfile` pour la creation de l'image du conteneur qui contiendra l'application `tomcat` dont le contenu est le suivant :

```
1 FROM ltm22/app-megasoft:1.0
2 COPY . /home/ubuntu/
3 ENTRYPOINT ["launch.sh"]
```

- Dans le fichier `catalina.sh` du dossier `bin` , modifier le variable `JAVA_HOME` :

```
1 JAVA_HOME=""
```

- Creez le fichier `launch.sh` dans le repertoire `bin` du projet qui contiendra le script qui sera lance au demarrge du conteneur permettant ainsi le lancement de Tomcat.

```
1 #!/bin/bash
2
3 echo "Go to /home/ubuntu/bin"
4
5 cd /home/ubuntu/bin
6
7 echo "Start mysql"
8
9 service mysql start
10
```

```
11 echo "Start Tomcat-Server"
12
13 /home/ubuntu/bin/catalina.sh run
```

- Dans le fichier de configuration de la BD, renseignez ceci :

```
1 database:3306,root,manager10,bdrsupreprodv1
```

car `database` sera le nom du conteneur de la BD.

2. Creation du fichier docker-compose.yml

Le service docker-compose est responsable de l'orchestration des conteneurs permettant ainsi à ces derniers de communiquer au sein d'un réseau isolé du reste du système. Ceci étant réalisé en créant un fichier docker-compose.yml dans le répertoire racine du projet dans la machine local. Son contenu doit être le suivant :

```
1 version: '2.18.1'
2
3 services:
4
5   application:
6     build:
7       context: .
8     container_name: application
9     restart: always
10    command: /bin/bash
11    tty: true
12    ports:
13      - 8080:8080
14    volumes:
15      - web:/home/ubuntu
16    networks:
17      - app-network
18
19  db:
20    build:
21      context: ./data
22    container_name: database
23    restart: always
24    tty: true
25    volumes:
26      - database:/var/lib/mysql
27    networks:
28      - app-network
29
30 volumes:
31   web:
32     driver: local
33     driver_opts:
34       o: 'bind'
35       type: 'none'
36       device: '/'
37   database:
38     driver: local
39     driver_opts:
40       o: 'bind'
41       type: 'none'
42       device: '/database'
```

```
43
44 networks:
45   app-network:
46     driver: bridge
47
```

⚠ Assurez-vous que le port défini dans votre application tomcat soit celui que vous avez fait passer dans le fichier `docker-compose.yml` (ligne 17). Vous pouvez choisir le port que vous souhaitez.

ℹ Pour plus d'informations sur les différents éléments de ce fichier, vous pouvez consulter la documentation de [docker-compose](#) ou faire un mail à cette adresse email: loicmeyong17@gmail.com.

Comme vous pouvez le constater dans le fichier `docker-compose.yml` il y a la section volumes qui permet de partager les données entre l'hôte local et le conteneur et d'assurer la persistance de ces données en cas de suppression des conteneurs. Vous avez les volumes :

- `web` (ligne 43) : qui envoie tout le contenu du projet dans le dossier home du conteneur afin que le projet soit lancé dans le conteneur. Donc à chaque modification du projet, vous n'aurez qu'à exécuter la commande :

```
1 $ docker-compose restart
```

- `database` (ligne 45) : qui récupère tout le contenu du répertoire `/var/lib/mysql` ou est contenu toutes les données de la BD conteneurisée et ainsi conserver le contenu de la base de données même en cas de destruction de ce conteneur. Ce volume est créé à partir du dossier `database` créé en amont dans le projet (ligne 54)

ℹ Pour plus d'informations sur les volumes docker, rendez vous sur [la documentation](#).

3. Lancement des conteneurs

Après avoir fait toutes les modifications ci-dessus, lancez la commande ci-dessous pour lancer la création des différents conteneurs :

```
1 $ cd repertoire-projet
2 $ docker-compose up -d --build
```

ℹ Avant de lancer cette commande, assurez vous d'être placé au niveau du répertoire racine du projet.

À chaque modification du projet, vous lancerez la commande :

```
1 $ docker-compose restart
```

ℹ L'argument `--build` est omis car la construction des images est déjà effectuée :

4. Envoie des images sur le hub

Une fois votre application fonctionnelle en local, publiez les images des différents services sur le hub afin que depuis le serveur de production les applications soient encapsulés dans les conteneurs lancés. Ceci se fait par :

```
1 $ docker tag nomDeLimageTomcat ltm22/app-megasoft
2 $ docker tag nomDeLimageBaseDeDonnees ltm22/db-megasoft
3 $ docker push ltm22/app-megasoft
4 $ docker push ltm22/db-megasoft
```

ℹ Avant de faire les opérations `docker push`, faites d'abord `docker login` afin de créer un canal de communication avec le hub distant. Pour vous, utilisez : username : ltm22 et password : designer155

5. Installation de docker et docker-compose sur le VPS

Depuis le serveur de production installer docker et docker-compose

```
1 $ apt-get update
2 $ apt-get install docker.io -y
3 $ apt-get install docker-compose -y
```

6. Deploiement dans le VPS

Depuis le serveur de production, creez :

- Le fichier `docker-compose.yml` dans le repertoire de l'utilisateur courant (Si c'est `ubuntu` c'est le dossier `/home/ubuntu`) dont le contenu sera :

```
1  version: '2.18.1'
2
3  services:
4
5    application:
6      image: ltm22/app-megasoft
7      container_name: application
8      restart: always
9      command: /bin/bash
10     tty: true
11     ports:
12       - 8080:8080
13     volumes:
14       - mdalsoft:/home/ubuntu/bin/mdalsoft
15       - uploadedfiles:/home/ubuntu/bin/uploadedfiles
16       - scripts:/home/ubuntu/bin/scripts
17     networks:
18       - app-network
19
20   db:
21     image: ltm22/db-megasoft
22     container_name: database
23     restart: always
24     tty: true
25     volumes:
26       - database:/var/lib/mysql
27     networks:
28       - app-network
29
30   volumes:
31     database:
32       driver: local
33       driver_opts:
34         o: 'bind'
35         type: 'none'
36         device: '/home/ubuntu/database'
37     mdalsoft:
38       driver: local
39       driver_opts:
40         o: 'bind'
41         type: 'none'
42         device: '/home/ubuntu/mdalsoft'
43     uploadedfiles:
```

```
44 driver: local
45 driver_opts:
46   o: 'bind'
47   type: 'none'
48   device: '/home/ubuntu/uploadedfiles'
49 scripts:
50 driver: local
51 driver_opts:
52   o: 'bind'
53   type: 'none'
54   device: '/home/ubuntu/scripts'
55
56 networks:
57   app-network:
58     driver: bridge
59
```

- Créez le dossier `database` dans le répertoire de l'utilisateur courant. Il sera utilisé pour la persistance des données de la BD conteneurisée.
- Créez le dossier `mdalsoft` dans le répertoire de l'utilisateur courant. Il sera utilisé pour la persistance des données du dossier `/bin/mdalsoft`.
- Créez le dossier `bin/uploadedfiles` dans le répertoire de l'utilisateur courant. Il sera utilisé pour la persistance des données du dossier `/bin/uploadedfiles`.
- Créez le dossier `bin/scripts` dans le répertoire de l'utilisateur courant. Il sera utilisé pour la persistance des données du dossier `/bin/script`.

Pour finaliser le déploiement, suivez le même procédé à partir de la partie 3.